

UNIVERSIDAD DEL VALLE DE GUATEMALA



Proyecto 2

Andre Marroquin Tarot - 22266

Sergio Orellana - 221122

Computación paralela y distribuida

Parte A:

DES Proceso de cifrado y descifrado

Datos de entrada (tamaños)

- **Bloque de datos:** 64 bits.
- **Clave:** 64 bits (56 útiles + 8 de paridad).
- **Subclaves:** 16 subclaves $K_1..K_{16}K_1..K_{16}K_1..K_{16}$ de 48 bits.

Glosario

- **PC-1:** Elección Permutada 1. Quita paridad y reordena la clave (64→56).
- **C0, D0:** Mitades de 28 bits que resultan de PC-1 (izq./der.).
- **PC-2:** Elección Permutada 2. Selecciona/reordena 48 bits de $C_i \parallel D_i \rightarrow K_i$
- **IP:** Permutación inicial al bloque de datos.
- **IP⁻¹:** Inversa de IP (permuta final).
- **L0, R0 / Li, Ri:** Mitades izquierda/derecha del bloque tras IP y tras la ronda i.
- **E(R):** Expansión de 32→48 bits para poder XORear con $K_iK_{-i}K_i$.
- **S-Boxes:** 8 tablas que convierten 6→4 bits (en total 48→32).
- **P:** Permutación que reordena los 32 bits que salen de las S-Boxes.
- **f(R,K):** Función de ronda = $P(S(E(R) \oplus K))$.
- $\oplus$: XOR (o exclusivo).
- $\parallel$: Concatenación de bits.

Cifrado (plaintext → ciphertext)

1. **Key schedule**
 - 1.1 PC-1: clave 64→56; dividir en C0 y D0 (28/28).
 - 1.2 Para $i=1..16$: rotar C_{i-1}, D_{i-1} (1 o 2 bits) → C_i, D_i .
 - 1.3 PC-2 sobre $C_i \parallel D_i \rightarrow K_i$ (48 bits).
2. **IP** al bloque de 64 bits.
3. **Dividir** en L0 y R0 (32/32).
4. **Rondas Feistel (i=1..16):**
 - 4.1 $X = E(R_{i-1}) \oplus K_i$ (32 → 48, XOR) .
 - 4.2 Pasar X por S-Boxes → 32 bits.

4.3 P sobre esos 32 bits.

4.4 **Actualizar mitades:**

- $L_i = R_{i-1}$

- $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$.

5. **Intercambio final:** formar $R_{16} || L_{16}$.

6. $IP^{-1} \rightarrow$ bloque cifrado de 64 bits.

Descifrado (ciphertext $\rightarrow$ plaintext)

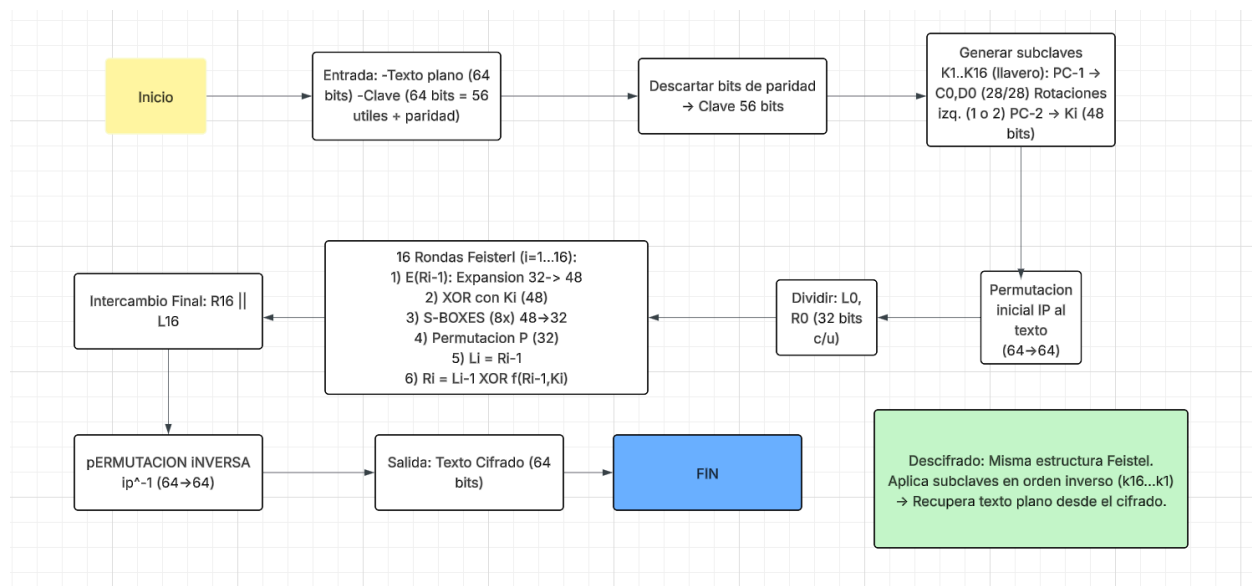
1. IP al bloque cifrado; dividir en L_0, R_0 .

2. **Rondas Feistel ($i=16..1$):** repetir los pasos 4.1–4.4 pero usando las subclaves en orden **inverso**: $K_{16}, K_{15}, \dots, K_1$.

3. **Intercambio final:** $R_{16} || L_{16}$.

4. $IP^{-1} \rightarrow$ texto plano original.

Diagrama de Flujo



Inciso 3

```

and@AM-DELL: ~/PY2-Paralela
and@AM-DELL:~/PY2-Paralela$
and@AM-DELL:~/PY2-Paralela$ PLAIN="hola andré cómo estás"
and@AM-DELL:~/PY2-Paralela$ echo "$PLAIN"
hola andré cómo estás
and@AM-DELL:~/PY2-Paralela$ B=22
KEY=$(( (1<<B) - 1 ))
and@AM-DELL:~/PY2-Paralela$ CIPH=$(./bin/bruteforce_seq --mode TEST --plain "$PLAIN" --key "$KEY" | sed -n 's/^\[TEST\] CIPH_HEX=//p')
and@AM-DELL:~/PY2-Paralela$ echo "CIPH_HEX_B${B}_WORST=$CIPH"
CIPH_HEX_B22_WORST=0AEAE57EB23A3A5375CE82F379E2923ACCD0E0E9E50F018
and@AM-DELL:~/PY2-Paralela$ taskset -c 0 /usr/bin/time -v \
./bin/bruteforce_seq --mode BRUTE --cipher-hex "$CIPH" --search "andré" --bits $B
[BRUTE] ENCONTRADA=1 KEY=4128510
[BRUTE] TIEMPO=1.194306 s RANGO=[0..4194303] ITER=4128511
[BRUTE] TEXTO="hola andré cómo estás"
Command being timed: "./bin/bruteforce_seq --mode BRUTE --cipher-hex 0AEAE57EB23A3A5375CE82F379E2923ACCD0E0E9E50F018 --search andré --bits 22"
User time (seconds): 1.32
System time (seconds): 0.00
Percent of CPU this job got: 111%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:01.19
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 3284
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 197
Voluntary context switches: 1
Involuntary context switches: 2
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
and@AM-DELL:~/PY2-Paralela$ |

```

```

and@AM-DELL: ~/PY2-Paralela
and@AM-DELL:~/PY2-Paralela$ OUTCSV="tests/seq_hola_andre_worst.csv"
echo "scenario,bits,key,iter,time_s" > "$OUTCSV"
and@AM-DELL:~/PY2-Paralela$ for B in 18 20 22 24; do
KEY=$(( (1<<B) - 1 ))
CIPH=$(./bin/bruteforce_seq --mode TEST --plain "$PLAIN" --key "$KEY" | sed -n 's/^\[TEST\] CIPH_HEX=//p')

OUT=$(./bin/bruteforce_seq --mode BRUTE --cipher-hex "$CIPH" --search "andré" --bits $B)

# EXTRAEMOS DE LA SEGUNDA LINEA: "[BRUTE] TIEMPO=... s ... ITER=..."
LINE=$(printf "%s\n" "$OUT" | grep '^[BRUTE] TIEMPO=')
T=$(printf "%s\n" "$LINE" | sed -n 's/^\[BRUTE\] TIEMPO=\([0-9.]\+\) s .*/\1/p')
I=$(printf "%s\n" "$LINE" | sed -n 's/^\.* ITER=\([0-9]\+\)\$/\1/p')

echo "WORST,$B,$KEY,{I:-NA},{T:-NA}" >> "$OUTCSV"
done
and@AM-DELL:~/PY2-Paralela$ column -s, -t "$OUTCSV"
scenario bits key iter time_s
WORST 18 262143 196351 0.062345
WORST 20 1048575 982783 0.316511
WORST 22 4194303 4128511 1.612024
WORST 24 16777215 16711423 5.320939
and@AM-DELL:~/PY2-Paralela$ OUTCSV2="tests/seq_hola_andre_avg.csv"
echo "scenario,bits,key,iter,time_s" > "$OUTCSV2"
and@AM-DELL:~/PY2-Paralela$ for B in 18 20 22 24; do
KEY=$(( (1<<(B-1)) ))
CIPH=$(./bin/bruteforce_seq --mode TEST --plain "$PLAIN" --key "$KEY" | sed -n 's/^\[TEST\] CIPH_HEX=//p')

OUT=$(./bin/bruteforce_seq --mode BRUTE --cipher-hex "$CIPH" --search "andré" --bits $B)

LINE=$(printf "%s\n" "$OUT" | grep '^[BRUTE] TIEMPO=')
T=$(printf "%s\n" "$LINE" | sed -n 's/^\[BRUTE\] TIEMPO=\([0-9.]\+\) s .*/\1/p')
I=$(printf "%s\n" "$LINE" | sed -n 's/^\.* ITER=\([0-9]\+\)\$/\1/p')

echo "AVG,$B,$KEY,{I:-NA},{T:-NA}" >> "$OUTCSV2"
done
and@AM-DELL:~/PY2-Paralela$ column -s, -t "$OUTCSV2"
scenario bits key iter time_s
AVG 18 131072 131073 0.053451
AVG 20 524288 524289 0.140437
AVG 22 2097152 2097153 0.490245
AVG 24 8388608 8388609 2.054050
and@AM-DELL:~/PY2-Paralela$

```

scenario	bits	key	iter	time_s
WORST	18	262143	196351	0.062345
WORST	20	1048575	982783	0.316511
WORST	22	4194303	4128511	1.612024
WORST	24	16777215	16711423	5.320939

Tabla Promedio

scenario	bits	key	iter	time_s
AVG	18	131072	131073	0.053451
AVG	20	524288	524289	0.140437
AVG	22	2097152	2097153	0.490245
AVG	24	8388608	8388609	2.05405

Método: Para cada B (18, 20, 22, 24) se generó un cifrado con la clave diseñada y medimos la búsqueda secuencial. WORST usa $2^B - 1$; AVG usa $2^{(B-1)}$ como llaves.

Hallazgo clave: En WORST el contador no llega exactamente a 2^B , sino a $2^B - 65,793$ (= 0x010101). Esto es por el ajuste de paridad por byte de DES (OpenSSL ajusta a paridad impar), lo que desplaza la clave efectiva hacia atrás en 0x010101.

Crecimiento: WORST crece exponencial con B ($O(2^B)$); AVG la mitad del rango ($\approx 2^{(B-1)}$), como se espera teóricamente.

Validación del mensaje: El texto descifrado contiene “hola andré cómo estás”, comprobando que el brute force funciona.

- **scenario:** condición de prueba (WORST = clave al final del rango; AVG = clave a la mitad).
- **bits (B):** tamaño del espacio de búsqueda (se prueban llaves $0..2^B-1$).
- **key:** llave entera con la que se generó el cifrado para esa prueba.
- **iter:** número de intentos realizados hasta encontrar la clave (incluye el intento ganador). En AVG: $\text{iter} \approx 2^{(B-1)}+1$. En WORST: $\text{iter} \approx 2^B$, pero baja 65,793 por el ajuste de paridad de DES.

- **time_s:** tiempo total de búsqueda medido por el programa; crece aproximadamente proporcional a iter (WORST $O(2^B)$, AVG $O(2^{(B-1)})$).

Inciso 4:

Se hizo funcionar el bruteforce.c con capa de compatibilidad con des_compat, que viene reemplazando a rpc/descript.h mediante OpenSSL

```
and@AH-DELL:~/PY2-Paralela$ mpicc -O3 -std=c11 -Iinclude src/bruteforce.c src/des_compat.c -o bin/bruteforce_mpi -lcrypto -DHAVE_OPENSSL -Wno-deprecated-declarations
and@AH-DELL:~/PY2-Paralela$ mpirun -n 4 ./bin/bruteforce_mpi

=====
brute-force mpi result (master rank 0)
=====
found_key : 816140
plaintext : "Save the planet "
=====
816140 l0A?}{B0X"F0
=====
brute-force mpi result (master rank 0)
=====
found_key : 816140
plaintext : "Save the planet "
=====
816140 l0A?}{B0X"F0
=====
brute-force mpi result (master rank 0)
=====
found_key : 816140
plaintext : "Save the planet "
=====
816140 l0A?}{B0X"F0
=====
brute-force mpi result (master rank 0)
=====
found_key : 816140
plaintext : "Save the planet "
=====
816140 l0A?}{B0X"F0
and@AH-DELL:~/PY2-Paralela$ |
```

Clave encontrada “Save the planet ”

Inciso 5:

a) decrypt(key, *ciph, len) y encrypt(key, *ciph, len)

Están sobre el backend des_compat_* (OpenSSL).

- **Entrada:** key entero con 56 bits efectivos, ciph buffer binario, len múltiplo de 8.
- **Salida:** escriben el resultado en el mismo ciph, usando un buffer temporal para evitar aliasing.

Diagrama:

Key (unit64) → 8 bytes + Paridad DES → DES-Key-Schedule

Ciph → [bloques de 8B] → DES-ECB(enc/dec) → tmp → copia a Ciph

b) try_key(key, *ciph, len)

Prueba una clave candidata y decidir si sirve buscando una marca en el texto plano. En este caso la marca es search_str = " the " .

Flujo

1. Clona el cifrado a un buffer temporal.
2. Descifra con decrypt(key, temp, len).
3. Convierte el buffer a C-string poniendo temp[len]=0.
4. Busca la subcadena con strstr.
5. Devuelve 1 si hay match; 0 si no.

Diagrama:

Key → decrypt (Key, ciph) → temp (texto)
temp → strstr (temp, search_str) → encontrado?

c) memcpy

Copia exactamente n bytes de src a dst sin preocuparse por el contenido. Es la herramienta correcta cuando no hay solapamiento.

Implementacion

- Copiar cifrado → temporal antes de descifrar (try_key).
- Copiar bloques de 8B dentro del backend DES.
- Copiar tmp → ciph para simular in-place seguro.

Diagrama:

`memcpy(dst, src, n)`

Si existiera solapamiento se usa memmove.

d) strstr

Busca la primera aparición de una subcadena en una cadena C.

Retorna puntero a la posición o NULL si no la encuentra.

Es la verificación de que la clave fue correcta: al descifrar, debe aparecer search_str dentro del texto plano.

Diagrama:

`return strstr(temp, search_str) != NULL;`

Iniciso 6:

Primitivas MPI:

bruteforce.c reparte el espacio de claves entre n_nodes procesos. Cada proceso prueba su rango y, si acierta, avisa a todos incluido él mismo para que el maestro imprima.

a) MPI_Irecv

Recepción no bloqueante. Inicia la recepción y vuelve de inmediato.

Todos los ranks postean un irecv para estar listos a recibir la clave cuando cualquiera la encuentre.

Codigo:

```
long recv_key = 0;
MPI_Irecv(&recv_key, 1, MPI_LONG, MPI_ANY_SOURCE, 0, comm, &req);
```

b) MPI_Send

Envío bloqueante. Retorna cuando el runtime ya puede garantizar la entrega buffering o receptor listo.

Patron cuando un proceso encuentra la clave...

Codigo:

```
for (int node = 0; node < n_nodes; ++node) {
    MPI_Send(&found, 1, MPI_LONG, node, 0, MPI_COMM_WORLD);
}
```

c) MPI_Wait

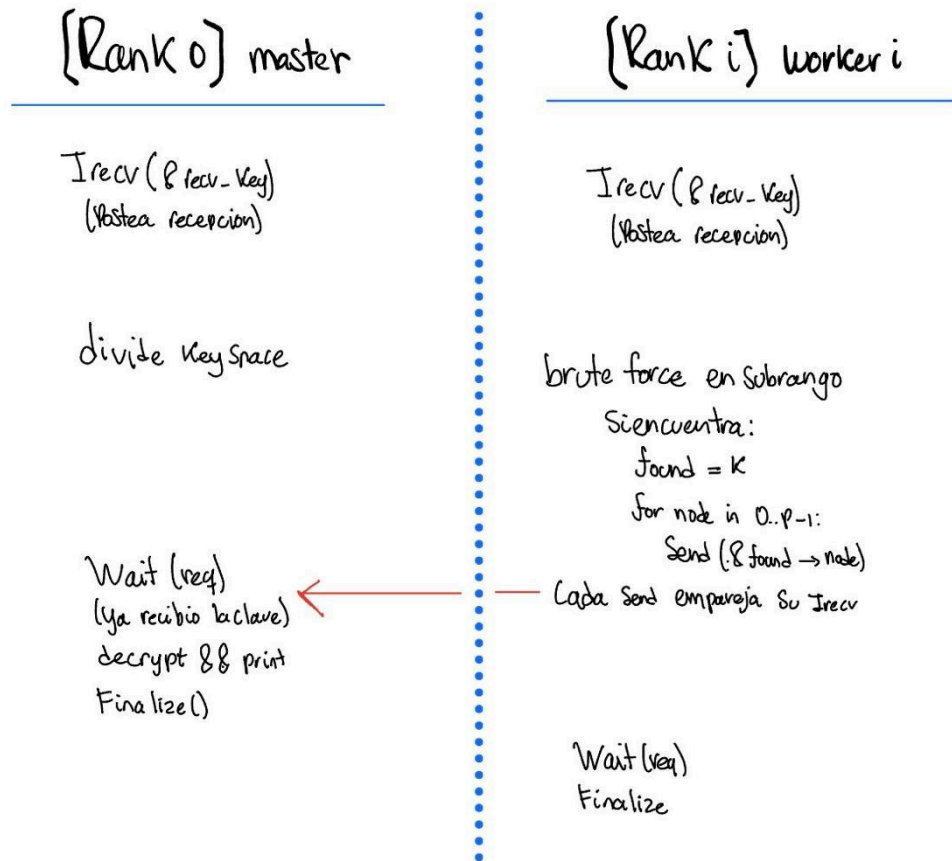
Bloquea hasta que una operación no bloqueante asociada a MPI_Request finalice.

Completar el irecv en todos los ranks antes de finalizar.

Codigo:

```
MPI_Wait(&req, &st);
if (id == 0) {
    Imprimir el resultado con la key aca
}
```

Flujo comunicacion:



Pseudocodigo flujo:

```

MPI_Irecv(&recv_key, 1, MPI_LONG, ANY_SOURCE, 0, comm, &req);

if (encontre) {
    for (node=0..n_nodes-1) MPI_Send(&found, 1, MPI_LONG, node, 0, comm);
}

MPI_Wait(&req, &st);

```

Parte B:

Evidencias:

Comandos:

```
and@AM-DELL: ~/PY2-Paralela$ make
cc -O3 -std=c11 -Wall -Wextra -Wshadow -Wpedantic -D_POSIX_C_SOURCE=200809L -Wno-deprecated-declarations -DHAVE_OPENSSL
-Iinclude -c src/des_compat.c -o build/des_compat.o
cc -O3 -std=c11 -Wall -Wextra -Wshadow -Wpedantic -D_POSIX_C_SOURCE=200809L -Wno-deprecated-declarations -DHAVE_OPENSSL
-Iinclude build/bruteforce_seq.o build/des_compat.o build/timer.o build/util.o -o bin/bruteforce_seq -lcrypto
cc -O3 -std=c11 -Wall -Wextra -Wshadow -Wpedantic -D_POSIX_C_SOURCE=200809L -Wno-deprecated-declarations -DHAVE_OPENSSL
-Iinclude -c src/file_crypto.c -o build/file_crypto.o
cc -O3 -std=c11 -Wall -Wextra -Wshadow -Wpedantic -D_POSIX_C_SOURCE=200809L -Wno-deprecated-declarations -DHAVE_OPENSSL
-Iinclude build/file_crypto.o build/des_compat.o -o bin/file_crypto -lcrypto
and@AM-DELL:~/PY2-Paralela$ ./bin/file_crypto --mode enc --in data/sample_plain.txt --out data/sample_cipher.hex --key 4
2
[enc] ok in='data/sample_plain.txt' out='data/sample_cipher.hex' key=42 bytes_in=48 bytes_enc=56
and@AM-DELL:~/PY2-Paralela$ ./bin/file_crypto --mode dec --in data/sample_cipher.hex --out data/sample_decrypted.txt --k
ey 42
[dec] ok in='data/sample_cipher.hex' out='data/sample_decrypted.txt' key=42 bytes_in=56 bytes_dec=48
and@AM-DELL:~/PY2-Paralela$
```

Texto plano:

```
file_crypto.c  Makefile  sample_plain.txt  des_compat.c  des_compat.h
data > sample_plain.txt
1 hola andré, cómo estás? este es un test DES.
2 |
```

Encriptado:

```
file_crypto.c  Makefile  sample_cipher.hex  des_compat.c  des_compat.h
data > sample_cipher.hex
1 FA8C9DF7BAE268718D90C4A25B46925283100B0C1C08C61C31117765C9BD82158BFD620520F79376B2888A71496F9E323AB385032C9DD17A
```

Desencriptado:

```
file_crypto.c  Makefile  sample_decrypted.txt  des_compat.c  des_compat.h
data > sample_decrypted.txt
1 hola andré, cómo estás? este es un test DES.
2 |
```

Referencias:

- Msmk. (2024, 10 septiembre). *¿Qué es el DES?* MSMK.

<https://msmk.university/que-es-el-des-msmk-university/>

- IBM Semeru Runtime Certified Edition for z/OS. (2025).

<https://www.ibm.com/docs/es/semeru-runtime-ce-z/21.0.0?topic=differences-data-encryption-standard-des-keys-operations>

